

# Programmazione GPU con CUDA

## Dalla CPU alla GPU: cambio di paradigma

La GPU nasceva esclusivamente come coprocessore dedicato al rendering grafico. Tuttavia, vista l'immensa potenza di calcolo parallelo, a partire dal 2005 è iniziata l'era del GP-GPU (general purpose GPU), trasformando queste schede in acceleratori per calcoli generici, arrivano a rimuovere le uscite video dai modelli dedicati ai datacenter. NVIDIA domina il mercato con l'architettura CUDA, dotata di un proprio compilatore ( `nvcc` ) e librerie dedicate.

### Perché la GPU è così veloce?

La differenza risiede nell'architettura dei transistor. Mentre una CPU impiega gran parte del suo silicio per logiche di controllo di flusso (branch prediction) e memorie cache molto capienti (per ridurre la latenza), la GPU è progettata per il calcolo *data parallel*. Sacrifica la logica di controllo complessa per massimizzare il numero di ALU. L'obiettivo della GPU non è ridurre la latenza di una singola operazione, ma massimizzare il *throughput* complessivo elaborando migliaia di dati contemporaneamente.

## Architettura Hardware vs. Astrazione Software

Per programmare in CUDA, dobbiamo comprendere come l'hardware reale venga mappato in astrazione software.

### Visione Hardware: SP, Warp e SM

- **Streaming Processor (SP):** è il "cuda core", l'unità fondamentale che esegue le istruzioni di un singolo thread.
- **Warp:** l'hardware non esegue i thread singolarmente in modo indipendente, ma li raggruppa in *warp* da 32 thread. Tutti i 32 thread di un warp eseguono esattamente la stessa istruzione nello stesso momento (paradigma SIMT - Single Instruction, Multiple Threads).
- **Streaming Multiprocessor (SM):** è il blocco costruttivo della GPU. Un SM contiene decine di SP (ad esempio 64 nelle architetture P100, V100 e A100), il Warp Scheduler (che decidono quale warp mandare in esecuzione), un'area dedicata ai registri (velocissima) privata per ogni scheduler e una cache L1/Memoria Condivisa. Una singola scheda GPU è formata da decine di SM (es. 108 su A100) che condividono un'enorme memoria globale.

### Visione Software: Thread, Block e Grid

Quando scriviamo il codice, non gestiamo direttamente i Warp o gli SM, ma utilizziamo un'astrazione gerarchica:

- **Thread:** l'unità atomica di calcolo.
- **Thread Block (blocco):** un raggruppamento di thread (massimo 1024). È un concetto importante, in quanto un intero blocco viene sempre assegnato a un singolo SM. Questo significa che i thread dello stesso blocco possono comunicare tra loro tramite la memoria condivisa dell'SM a latenza quasi nulla.
- **Grid (griglia):** l'insieme di tutti i blocchi lanciati da un kernel. I blocchi all'interno della griglia non hanno garanzie sull'ordine di esecuzione. Possono comunicare tra loro *solo* tramite la lenta Memoria Globale.

Le dimensioni di blocchi e griglie possono essere a 1, 2 o 3 dimensioni, gestite tramite il tipo di dato `dim3`. Per mappare i calcoli (es. la somma di array), ogni thread deve sapere "chi è". Lo fa usando le variabili integrate di CUDA:

- `threadIdx`: l'indice del thread nel blocco.

- `blockIdx` : l'indice del blocco nella griglia.

- `blockDim` : la dimensione del blocco.

La formula classica per l'indicizzazione 1D è: `idx = blockIdx.x * blockDim.x + threadIdx.x`.

## CUDA

CUDA è un framework per l'elaborazione parallela creato da NVIDIA per le proprie schede GPU.

CUDA Toolkit fornisce gli strumenti di sviluppo, librerie e la documentazione necessari per creare applicazioni per l'architettura CUDA.

SO supportati: Linux e Windows.

Linguaggi di programmazione disponibili nell'ambiente di sviluppo:

- CUDA-C (C con estensioni NVIDIA),
- estensioni di Python, Fortran, Java e MATLAB.

## Modello di Esecuzione e Qualificazioni

Il programma CUDA è eterogeneo: inizia e finisce sulla CPU (Host), la quale delega i calcoli pesanti alla GPU (Device).

I qualificatori di funzione dicono al compilatore per quale unità deve tradurre il codice:

- `__global__` : definisce una funzione GPU kernel, può essere invocata solo dall'host e viene eseguito sul device. Non ritorna valori (deve essere `void`). Viene lanciato con la sintassi `<<<grid, block>>>`.
- `__device__` : funzioni d'appoggio chiamate dal device ed eseguite sul device (tipo `void`).
- `__host__` : (default) funzioni standard CPU (eseguite ed invocate dall'host).

### Asincronia:

Quando l'host lancia un kernel, la chiamata è asincrona: la CPU non si ferma ad aspettare la GPU, ma prosegue subito alla riga di codice successiva. Questo permette di sovrapporre calcoli CPU e GPU. Se l'host deve aspettare i risultati per procedere, si deve invocare esplicitamente la funzione di barriera `cudaDeviceSynchronize()`. In alternativa, si genera un sincronismo implicito quando si richiede un `cudaMemcpy` classico per riportare i dati dal device all'host (la copia inizia solo dopo che il kernel ha terminato).

## Gerarchia delle Memorie (ottimizzazione)

Le performance in HPC si vincono o si perdono sulla gestione della memoria. CUDA offre diversi spazi di memoria:

1. **Registri:** memoria velocissima privata per ogni singolo thread. Qui finiscono le variabili scalari locali.
2. **Memoria locale:** fisicamente risiede nella memoria globale (anche se cachata), viene usata automaticamente dal compilatore per array locali di un thread troppo grandi per stare nei registri (tipicamente oltre i 4 elementi).
3. **Memoria condivisa:** dichiarata con `__shared__`. È una memoria velocissima on-chip, condivisa tra i thread di uno stesso blocco. Funziona come una cache gestita dal programmatore. Il pattern tipico è: i thread copiano i dati dalla global alla shared, li elaborano e li riscrivono nella global.
  - *Bank Conflict:* la shared memory è divisa in 32 banchi di memoria larghi 4 byte. Se due o più thread dello stesso Warp tentano di leggere indirizzi diversi che mappano sullo stesso banco, le letture vengono serializzate, distruggendo le performance.
4. **Memoria globale:** enorme, accessibile da tutta la griglia, ma con latenze altissime (centinaia di cicli di clock).

- **Coalescing:** per massimizzare la banda passante, gli accessi devono essere "coalesced" (raggruppati). Se i 32 thread di un warp leggono indirizzi di memoria globale adiacenti, l'hardware unisce le 32 letture in un'unica transazione (coalescing). Se invece ogni thread legge indirizzi distinti, l'hardware deve fare decine di costose transazioni separate. Per questo si usa `cudaMallocPitch()` sugli array 2D, che aggiunge del "padding" (byte vuoti a fine riga) per garantire che l'inizio di ogni riga sia perfettamente allineato in memoria per il coalescing.

5. **Memoria costante e texture:** la constant memory (64 KB) è ottimizzata con cache hardware specifiche ed è ideale quando tutti i thread devono leggere contemporaneamente uno stesso dato immutabile. La *texture memory* è ottima per accessi con coerenza spaziale 2D/3D (utile in elaborazione immagini).

## API di Gestione Memoria e Unified Memory

Il workflow tradizionale richiede tre step espliciti: allocazione su GPU con `cudaMalloc()`, trasferimento dati da host a device tramite `cudaMemcpy()` (direzionalità `cudaMemcpyHostToDevice`), esecuzione del kernel e trasferimento opposto `cudaMemcpyDeviceToHost`. Infine si libera con `cudaFree()`.

Dall'architettura Pascal (P100) e CUDA 6 in poi, è stata introdotta la **Unified Memory**. Allocando la memoria con `cudaMallocManaged()` o tramite il prefisso `__managed__`, CPU e GPU condividono lo stesso spazio di indirizzamento virtuale. Non serve più usare `cudaMemcpy`: ci pensa il driver a migrare fisicamente le pagine di memoria tra la RAM del PC e la VRAM della scheda video in modo automatico "sotto al cofano" ogni volta che un device richiede quel dato.

## Sincronizzazione e Condition Race

Essendo un ambiente iper-parallelo, bisogna gestire attentamente le *race condition*: l'ordine con cui i thread leggono o scrivono una variabile condivisa è intrinsecamente imprevedibile.

- **Sincronizzazione di blocco:** per fare aspettare tutti i thread di uno stesso blocco prima di procedere (ad esempio durante un algoritmo di riduzione logaritmica), si usa la barriera `__syncthreads()`. Funziona solo per i thread all'interno dello stesso blocco.
- **Operazioni atomiche:** per operazioni in cui più thread tentano di aggiornare una stessa variabile globale o condivisa contemporaneamente (es. contatori o somme), CUDA fornisce funzioni atomiche come `atomicAdd()`, `atomicMax()`, ecc.. Queste garantiscono che l'operazione di lettura-modifica-scrittura venga eseguita da un singolo thread in modo indivisibile.
- **Sezioni critiche con mutex:** per creare lock (sezioni critiche vere e proprie) si combinano funzioni come `atomicCAS` (Compare-and-Swap) e `atomicExch` insieme a `__threadfence()`, che forza la visibilità delle modifiche in memoria globale prima di sbloccare il lock per gli altri blocchi.